# Especificação Técnica e Plano de Desenvolvimento

Sistema de Detecção e Registro de Movimentação de Produtos por Visão Computacional para Pequenos Estabelecimentos

**Autora:** Judith de Andrade

*Aprofunda a proposta revisada da atividade anterior num plano executável. Fatos sobre datasets/modelos já verificados anteriormente (ver Referências); esta etapa traz decisões de especificação e planejamento, não nova pesquisa externa.*

## 1. Especificação do problema

**Tarefa:** detectar produtos pré-cadastrados em vídeo gravado de uma prateleira/balcão, rastrear a mesma unidade entre quadros, e registrar automaticamente quando ela cruza uma zona de saída definida.

**Usuário:** responsável (proprietário/operador) de um pequeno estabelecimento — mercado de bairro, balcão de restaurante — que hoje confere estoque manualmente.

**Entrada:** vídeo gravado da cena. **Saída:** eventos `{produto, timestamp, direção, confiança}`, num painel simples.

**Cenário de uso:** vídeo de teste pré-gravado (não câmera ao vivo), 3 a 5 produtos físicos numa prateleira/balcão fixo, simulando retiradas.

**Dentro do escopo:** detecção/classificação dos 3–5 produtos-alvo; rastreamento entre quadros (ByteTrack); regra de zona única; registro em banco local e painel de histórico.

**Fora do escopo:** câmera ao vivo; catálogo extenso; rastreamento/identificação de pessoas; reconhecimento facial (excluído por decisão de projeto, não só por prazo); pagamento real; múltiplas câmeras/zonas.

**Diferencial frente a tecnologias tradicionais:** soluções de mercado usam um sinal isolado — peso (célula de carga), RFID (etiqueta), óptico/IR (feixe) ou ESL (preço eletrônico) — cada uma exigindo hardware específico e resolvendo só parte do problema.

| Tecnologia | Sinal detectado | Limitação principal |
|---|---|---|
| Peso/carga | Variação de peso | Exige balança física; produtos de peso parecido confundem |
| RFID | Identidade da etiqueta | Exige etiquetar cada unidade + antenas |
| Visão computacional tradicional | Presença/posição do produto | Dificuldade com produtos sobrepostos/retirados juntos |
| Óptico/IR | Interrupção de feixe | Só funciona bem em ponto único (ex.: vending machine) |
| ESL | Preço/informação | Não é tecnologia de inventário por si só |

O SmartStock Vision propõe integrar detecção visual à contagem de quantidade, ao rastreamento da movimentação, à cesta virtual e à conferência de pagamento, identificando divergências entre o que foi retirado e o que foi pago. O fluxo de cesta virtual, estoque e pagamento **já está implementado e testado** no protótipo atual; a detecção visual treinada é a etapa em desenvolvimento que substituirá a simulação usada hoje.

## 2. Dados

**Fontes públicas (pré-treino): SKU-110K** (Goldman et al., 2019 — 11.743 imagens, ~1,7M caixas, classe única "produto") e **RPC** (Wei et al., 2019 — 30.000 cenas de checkout, ~200 classes) — usados via transfer learning, não como dataset final (classes diferentes das do MVP).

**Dataset próprio:** vídeo/fotos dos produtos-alvo gravados pela equipe. Meta: 150–300 imagens/classe (frames extraídos do vídeo em ângulos variados), JPG, mínimo 640×640px. **Preparação:** extração de frames (~1 a cada 0,5s), remoção manual de frames ruins, anotação em **Roboflow Annotate** (bounding box, revisão humana obrigatória mesmo com Auto Label), exportação em formato YOLO.

**Rótulos:** datasets públicos já rotulados; conjunto próprio requer anotação manual assistida.

**Riscos:** desbalanceamento entre produtos (mitigação: meta mínima igual por classe); viés de captura (mesmo ambiente/iluminação — limitação documentada); privacidade/LGPD (enquadramento restrito aos produtos; pessoa eventualmente capturada é tratada só como região genérica, nunca identificada); segurança (imagens próprias mantidas no repositório do grupo, sem publicação pública).

## 3. Abordagem de IA

**Principal: YOLOv8** (Ultralytics), transfer learning — pré-treinado no COCO, ajustado com SKU-110K e depois com o conjunto próprio.

**Alternativa: YOLO26** (2026, *edge-first*, sem NMS) — comparação de desempenho/velocidade se o tempo permitir, não base principal (menos documentação/comunidade, maior risco de travar sem solução encontrável).

**Baseline sem IA:** subtração de fundo via OpenCV — detecta mudança na cena, não identifica o produto.

**Rastreamento: ByteTrack** (mais leve que DeepSORT, sem rede extra de re-identificação — suficiente para zona única).

**Recursos:** GPU gratuita (Google Colab, ~15–30h/semana no nível gratuito, a reconfirmar). Inferência em vídeo gravado não exige tempo real.

**Adaptação:** fine-tuning (transfer learning); não é necessário treinar do zero nem usar prompting/RAG (tarefa é visão computacional supervisionada).

## 4. Arquitetura

Vídeo → captura de quadros → pré-processamento (OpenCV) → **YOLOv8** (caixa + classe + confiança) → **ByteTrack** (identidade entre quadros) → regra de zona (cruzou a borda definida?) → registro (SQLite) → painel (Streamlit, histórico + alerta).

Cada componente é testável isoladamente — o modelo pode ser avaliado sozinho, com métricas de detecção, antes de integrar a regra de zona e o painel, validando a viabilidade cedo no cronograma.

## 5. Protótipo mínimo (MVP)

**Essencial:** (1) YOLOv8 ajustado detectando os 3–5 produtos-alvo; (2) ByteTrack mantendo identidade entre quadros; (3) regra de zona única gerando evento de saída; (4) registro em SQLite; (5) painel Streamlit com a lista de eventos.

**Adiado:** câmera ao vivo, múltiplas zonas, catálogo extenso, notificação externa (e-mail/push), integração com pagamento.

*Capturas de tela do protótipo atual (cesta virtual, sensor de peso simulado, reconhecimento por foto) e um mockup ilustrativo da visão de produto final estão no anexo "Capturas de Tela".*

---

## 6. Plano experimental

**Perguntas a responder:**

1. O YOLOv8 ajustado supera os dois baselines (subtração de fundo; YOLOv8 sem ajuste)?
2. O ByteTrack mantém a identidade da unidade sob oclusão breve na cena de teste?
3. Qual a relação entre desempenho (F1/mAP) e custo (FPS)?
4. O YOLO26, se testado, compensa a documentação mais escassa com ganho real de desempenho/velocidade?

**Baselines:** subtração de fundo (OpenCV, sem IA); YOLOv8 pré-treinado sem fine-tuning.

**Métricas:** F1-score e mAP (comparáveis com a literatura), IoU (critério de acerto de caixa), FPS (viabilidade prática).

**Experimentos:** comparação de F1/mAP entre baselines e modelo proposto; FPS em CPU vs. GPU; robustez sob iluminação alterada (reportada como limitação, não escondida); amostra qualitativa de falsos positivos/negativos.

---

## 7. Critérios de sucesso

- Superar **ambos os baselines** em F1-score no conjunto de teste próprio.
- F1-score ≥ **0,40**, usando como âncora de calibração o resultado de Shoman et al. (2022) em problema semelhante (F1 = 0,44) — um valor próximo já é honesto e satisfatório dado o escopo de um semestre.
- FPS suficiente para processar o vídeo de teste em até **2× sua duração real** (ex.: vídeo de 5 min em até 10 min) — critério de viabilidade offline, já que tempo real está fora do MVP.
- ByteTrack não deve trocar identidade em mais de 1 a cada 10 eventos registrados (checagem manual, por amostragem).

Critério não atingido é reportado como resultado real, com hipótese de causa — não é escondido da avaliação.

## 8. Planejamento de desenvolvimento

| Etapa | Atividade |
|---|---|
| 1 | Definir produtos-alvo; gravar vídeo de teste |
| 2 | Extrair frames; anotar no Roboflow |
| 3 | Implementar baseline 1 (subtração de fundo) |
| 4 | Rodar baseline 2 (YOLOv8 sem ajuste) |
| 5 | Fine-tuning do YOLOv8 (Colab) |
| 6 | Avaliar modelo ajustado (F1, mAP, IoU) vs. baselines |
| 7 | Integrar ByteTrack + regra de zona |
| 8 | Implementar registro (SQLite) e painel (Streamlit) |
| 9 | Integração ponta a ponta; experimentos de FPS/robustez |
| 10 | Comparar com critérios de sucesso; documentar resultados |

## 9. Riscos e plano B

| Risco | Plano B |
|---|---|
| Poucos dados próprios | Reforçar pré-treino em SKU-110K/RPC; reduzir produtos-alvo (5→3) |
| GPU do Colab indisponível | Menos épocas/imagem menor; CPU para inferência |
| ByteTrack falha sob oclusão | Simplificar cena de teste; documentar limitação |
| F1 abaixo da meta | Reportar como resultado real, com hipótese de causa |
| Tempo insuficiente | Priorizar até a etapa 8; cortar 9–10 primeiro se faltar tempo |

## 10. Uso de IA

| Ferramenta | Atividade | Verificação |
|---|---|---|
| Claude | Estruturar a especificação a partir da investigação já validada | Nenhum dado novo de fonte externa nesta etapa — fatos técnicos já confirmados antes; decisões de planejamento (critérios, cronograma) são escolhas do grupo, não fatos a verificar |
| Claude | Definir critério F1 ≥ 0,40 | Âncora (F1=0,44, Shoman et al. 2022) vem de fonte já verificada; a meta em si é decisão do grupo |

## Referências


1. Goldman, E.; Herzig, R.; Eisenschtat, A.; Ratzon, O.; Levi, I.; Goldberger, J.; Hassner, T. **Precise Detection in Densely Packed Scenes.** CVPR 2019. https://arxiv.org/abs/1904.00853
2. Wei, X.-S.; Cui, Q.; Yang, L.; Wang, P.; Liu, L. **RPC: A Large-Scale Retail Product Checkout Dataset.** 2019. https://arxiv.org/abs/1901.07249
3. Shoman, M.; Aboah, A.; Morehead, A.; Duan, Y.; Daud, A.; Adu-Gyamfi, Y. **A Region-Based Deep Learning Approach to Automated Retail Checkout.** 2022. https://arxiv.org/abs/2204.08584
4. Ultralytics. **YOLO26 Documentation.** 2026. https://docs.ultralytics.com/models/yolo26
5. Ultralytics. **SKU-110K Dataset Documentation.** https://docs.ultralytics.com/datasets/detect/sku-110k
6. Roboflow. **Roboflow Annotate.** https://roboflow.com/annotate
7. Google. **Google Colaboratory.** (Consultar FAQ oficial mais recente para limites de GPU vigentes.) https://colab.research.google.com
8. BRASIL. **Lei nº 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais — LGPD).** http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm


*Documento complementar à "Proposta Revisada — Investigação e Aprofundamento Técnico" (atividade anterior), que traz a investigação completa de trabalhos relacionados, viabilidade e diagnóstico da proposta original.*